Capacitação Profissional em Microeletrônica | PADIS Programa de Apoio ao Desenvolvimento Tecnológico da Indústria de Semicondutores

O Programa de Apoio ao Desenvolvimento Tecnológico da Indústria de Semicondutores (PADIS)

Jacques Costa Brito
Rômulo da Silva Lira
Thiago dos Santos Alves
Vitor Carvalho dos Santos

RELATÓRIO – MULTIPLEXADOR (MUX) DE 4 ENTRADAS

MANAUS - AM
05 / 03 / 2026

Executores: iRede INOVAÇÃO E TECNOLOGIA | Patrocinador: Multi

SUMÁRIO

1. INTRODUÇÃO **3**
2. OBJETIVOS **4**
3. MATERIAIS E EQUIPAMENTOS UTILIZADOS **4**
4. RESULTADOS EXPERIMENTAIS **4**
5. CONCLUSÕES **9**
6. REFERÊNCIAS BIBLIOGRÁFICAS **10**

1. INTRODUÇÃO

> Hoje em dia, o termo *digital* tornou-se parte do nosso vocabulário diário devido ao modo intenso pelo qual os circuitos digitais e as técnicas digitais passaram a ser utilizados em quase todas as áreas: computadores, automação, robôs, tecnologia e ciência médica, transportes, telecomunicações, entretenimento, exploração espacial, e assim por diante [1].
> Diagramas de tempos são usados para descrever a operação detalhada de um sistema digital. As transições entre o nível de baixa tensão (quando a chave é solta) e o de alta tensão (quando a chave é pressionada) neste diagrama de tempos são traçadas como linhas verticais bem definidas [1].

"Um **sistema digital** é uma combinação de dispositivos projetados para manipular informação lógica ou quantidades físicas representadas no formato digital; ou seja, as quantidades podem assumir apenas valores discretos [1]".

"Um **sistema analógico** contém dispositivos que manipulam quantidades físicas representadas na forma analógica. Em um sistema analógico, as quantidades físicas podem variar ao longo de uma faixa contínua de valores [1]".

> No sistema binário, há apenas dois símbolos ou valores possíveis para os dígitos: 0 e 1. Mesmo assim, também pode ser usado para representar qualquer quantidade que possa ser representada em decimal ou em outro sistema de numeração. Entretanto, é comum que o sistema binário use um número maior de dígitos para expressar determinado valor [1].

"A álgebra booleana não é usada apenas como instrumento de análise e simplificação de sistemas lógicos; é também uma valiosa ferramenta de projeto usada para que um circuito lógico produza uma relação entrada/saída [1]".

"As entradas são consideradas variáveis lógicas cujos níveis lógicos determinam, a qualquer momento, os níveis da(s) saída(s). Como os valores possíveis de uma variável são apenas dois, a álgebra booleana é mais fácil de ser manipulada que a álgebra convencional [1]".

"O *multiplexador digital*, ou *seletor de dados*, é um circuito lógico que recebe diversos dados digitais de entrada e seleciona um, em determinado instante, para transferi-lo para a saída [1]".

> O envio do dado de entrada desejado para a saída é controlado pelas entradas de SELEÇÃO (frequentemente denominadas ENDEREÇO). O multiplexador atua como uma chave de múltiplas posições controlada digitalmente, em que o código aplicado nas entradas de SELEÇÃO controla a entrada de dados que será comutada para a saída [1].

2. OBJETIVOS

O objetivo desta tarefa é projetar e verificar um multiplexador (MUX) de 4 entradas para 1 saída utilizando a linguagem SystemVerilog. Por meio desta atividade, será consolidado o entendimento da estrutura de módulos, declaração de portas, tipos de dados, operadores lógicos e a aplicação de blocos procedurais (always_comb) ou atribuições contínuas (assign) para descrever lógica combinacional. Além disso, será desenvolvido um testbench para validar o comportamento do MUX, utilizando a plataforma online EDA Playground.

3. MATERIAIS E EQUIPAMENTOS UTILIZADOS

Foram utilizados computadores com o sistema operacional Windows 10 e Windows 11.

4. RESULTADOS EXPERIMENTAIS

Um Multiplexador 4-para-1 é um circuito combinacional que seleciona uma de quatro entradas de dados (D_0, D_1, D_2, D_3) e a direciona para uma única saída (Y), com base no valor de duas linhas de seleção (S_1, S_0). A funcionalidade esperada é que:

- Se $2'b00$, $Y = D_0$
- Se $2'b01$, $Y = D_1$
- Se $2'b10$, $Y = D_2$
- Se $2'b10$, $Y = D_3$

Implementar este circuito em SystemVerilog e, em seguida, criar um *testbench* para simular e verificar todas as possíveis combinações de entradas e seleções, garantindo que o MUX funcione corretamente.

Para a implementação deste circuito, utilizou-se a linguagem de descrição de hardware (HDL) SystemVerilog, consolidando assim o entendimento da estrutura modular da ferramenta de simulação. Como boa prática de projeto microeletrônico, o módulo mux_4_to_1 foi desenvolvido de forma genérica e parametrizável.

Isso foi feito declarando a variável lógica de barramento com uma constante temporal em escopo através do uso de parameter DATA WIDTH = 8, preestabelecida com um limite padrão de operação em base de 8 bits para cada canal.

O roteamento condicional que amarra as entradas foi resolvido abstratamente (nível RTL) na declaração interna do módulo usando a estrutura de controle dinâmico baseada em hardware case, envolto rotineiramente em um escopo dinâmico avaliado por meio da diretiva always_comb – garantindo assim avaliações instantâneas nas saídas face à qualquer flutuação dos níveis eletrônicos lógicos de entrada, evitando as latências acidentais e o engessamento típico da construção assign em blocos múltiplos.

A Figura 1 a seguir ilustra a descrição do código do multiplexador 4-para-1.

Figura 1 – Montagem do circuito referente a expressão.

```
// ---------------------------------------------------------------------------
// Módulo: mux_4_to_1
// Descrição: Multiplexador 4-para-1 parametrizável em SystemVerilog.
// ---------------------------------------------------------------------------
module mux_4_to_1 #(
    parameter DATA_WIDTH = 8
)(
    input  logic [DATA_WIDTH-1:0] D0,
    input  logic [DATA_WIDTH-1:0] D1,
    input  logic [DATA_WIDTH-1:0] D2,
    input  logic [DATA_WIDTH-1:0] D3,
    input  logic [1:0] S,
    output logic [DATA_WIDTH-1:0] Y
);

    // Bloco procedural combinacional para descrever o comportamento do MUX
    always_comb begin
        case (S)
            2'b00: Y = D0;
            2'b01: Y = D1;
            2'b10: Y = D2;
            2'b11: Y = D3;
            default: Y = '0; // Boa prática: definir um valor padrão
        endcase
    end

endmodule

// ---------------------------------------------------------------------------
// Testbench: tb_mux_4_to_1
// Descrição: Arquivo de teste para simular e validar o mux_4_to_1.
// ---------------------------------------------------------------------------
module tb_mux_4_to_1;

    // Definição do parâmetro
    localparam DATA_WIDTH = 8;

    // Declaração dos sinais internos para conectar ao módulo
    logic [DATA_WIDTH-1:0] D0;
    logic [DATA_WIDTH-1:0] D1;
    logic [DATA_WIDTH-1:0] D2;
    logic [DATA_WIDTH-1:0] D3;
    logic [1:0] S;
    logic [DATA_WIDTH-1:0] Y;

    // Instanciação do MUX 4-para-1 (Device Under Test - DUT)
    mux_4_to_1 #(
        .DATA_WIDTH(DATA_WIDTH)
    ) dut (
        .D0(D0),
        .D1(D1),
        .D2(D2),
        .D3(D3),
        .S(S),
        .Y(Y)
    );
```

Fonte: Autores (2026).

Comprovada a compilação limpa do Módulo Base sem erros estruturais, elaborou-se no mesmo projeto do código-fonte um arcabouço para instanciar o Mux e validar a sua integridade face à comutação de chaves – o Testbench (tb_mux_4_to_1).

Para garantir que a seleção das entradas pelo barramento S (seletores de 00 a 11) ocorreria corretamente em todas as malhas de simulação propostas, injetou-se sinais digitais simuladores estáticos na malha de entrada, distinguindo com precisão empírica cada uma destas com valores de arquitetura base em representação hexadecimal: $D_0 \leftarrow AA$, $D_1 \leftarrow BB$, $D_2 \leftarrow CC$ e $D_3 \leftarrow DD$.

Invocando o ambiente da função procedural sequenciada de testes initial begin, promoveu-se uma escadaria de incrementações cadenciadas sobre a chave S (através do operador de atraso da simulação #10 a cada step). Adicionou-se mecanismos de escrutínio para o log do terminal atrelados à premissa do comportamento, como demonstra o resultado abaixo em shell:

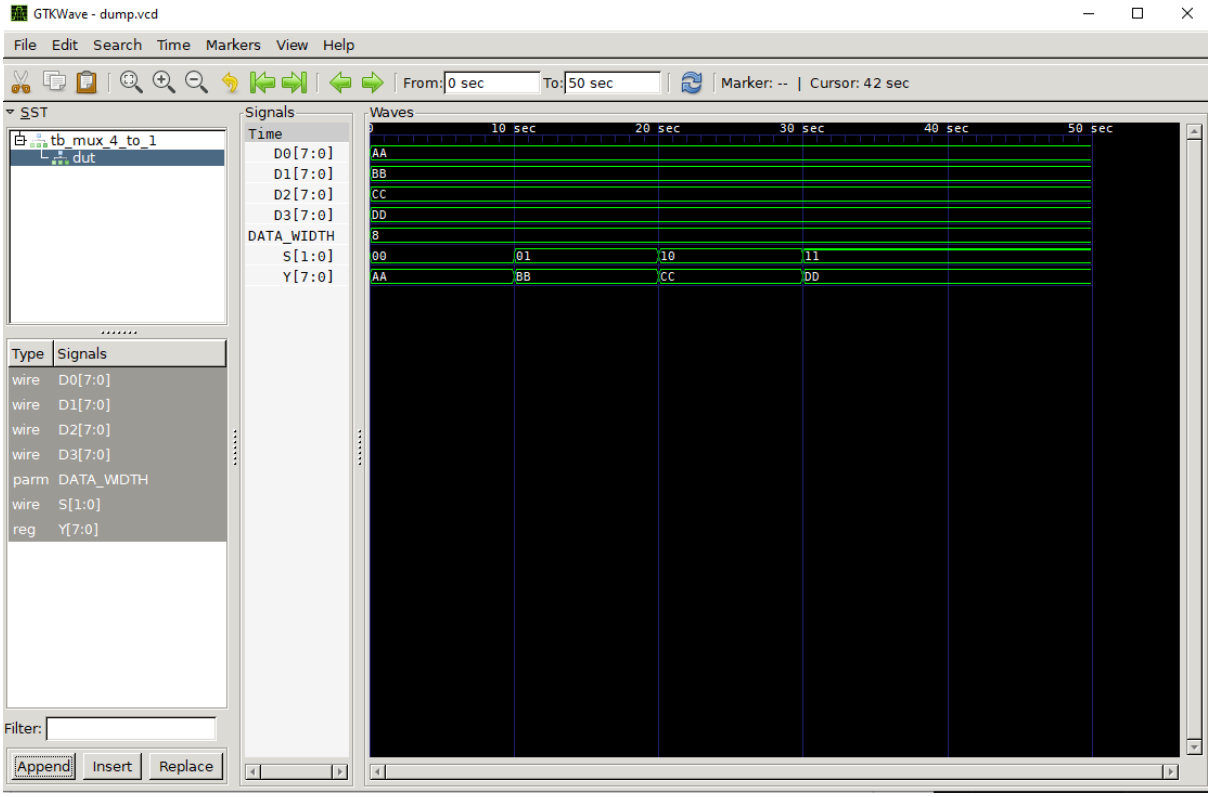
```
Iniciando Simulação do MUX 4-para-1...
-----------------------------------------------
Tempo=10 | Sel(S)=00 | Saida(Y)=aa -> Esperado=aa
Tempo=20 | Sel(S)=01 | Saida(Y)=bb -> Esperado=bb
Tempo=30 | Sel(S)=10 | Saida(Y)=cc -> Esperado=cc
Tempo=40 | Sel(S)=11 | Saida(Y)=dd -> Esperado=dd
-----------------------------------------------
Simulação concluida!
```

O uso das funções de depuramento automático – como error() em blocos condicionais comparativos (Y !== Dn), não acusaram nenhuma falha durante a execução técnica da comutação analítica, provando textualmente o rigor e sucesso do circuito.

Visando corroborar com os dados extraídos pelo console e garantir uma inspeção a nível visual do chaveamento elétrico no domínio do tempo, os vetores simulados encapsulados no arquivo *Value Change Dump* (dump.vcd) que acompanharam a diretiva final foram consumidos no visualizador e sintetizador lógico de ondas GTKWave.

O gráfico atestou o chaveamento estático das vias em escala macro, o qual é ilustrado na Figura 2 abaixo.

Figura 2 – Formas de onda do multiplexador 4-para-1.



Fonte: Autores (2026).

5. CONCLUSÕES

O relatório em questão abordou o multiplexador, um componente de eletrônica digital que é utilizado para selecionar uma determinada saída baseado em uma seleção da entrada. Foi realizada a montagem lógica desse multiplexador, na configuração 4-para-1, com as suas respectivas entradas e saída, e então realizada a simulação do mesmo.

Com base nos resultados obtidos nas simulações, foi possível comprovar o correto funcionamento do elemento do circuito.

6. REFERÊNCIAS BIBLIOGRÁFICAS

[1] TOCCI, Ronald J. Sistemas digitais : princípios e aplicações. 11. Ed. São Paulo: Pearson Prentice Hall, 2011.